

p1053R0

Future-proofing continuations for executors

Published Proposal, 6 May 2018

This version:

<https://wg21.link/P1053>

Authors:

[Lee Howes](#) (Facebook)

[Eric Niebler](#) (Facebook)

Audience:

SG1, LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

https://github.com/LeeHowes/CPP/blob/master/future_continuations.bs

Table of Contents

1	Introduction
2	Requirements
2.1	then_value
2.2	then_error
2.3	then_variant
2.4	then_value_with_exception_log
3	Concept
4	Examples
4.1	then_value
4.2	then_error
4.3	then_variant

4.4 `then_value_logging_error`

5 **Noexcept**

6 **Rethinking bulk execution**

7 **Proposed New Wording for P0443**

7.1 `Promise` requirements

7.2 `OneWayFutureContinuation` requirements

7.3 `TwoWayFutureContinuation` requirements

7.4 `ThenFutureContinuation` requirements

7.5 Changes to `OneWayExecutor` requirements

7.6 Changes to `TwoWayExecutor` requirements

7.7 Changes to `ThenExecutor` requirements

7.8 Changes to Bulk requirements in general

Contributors:

- Jay Feldblum
- Andrii Grynenko
- Kirk Shoop

§ 1. Introduction

[p0443](#) defines interfaces for executors and the continuation functions passed to them. [p1054](#) utilises these fundamental interfaces to build expressive concepts for future types where continuations are cleanly mapped through continuation construction functions.

The current design of the continuation functions passed to `then_execute` are based on the ability of the executor to invoke the continuation.

In essence the continuations have an interface similar to:

```
struct callable {  
    R operator()(T);  
    R operator()(exception_arg, e);  
};
```

where either function is optional, and in that situation the other operation will act as a passthrough. One reason for designing the API in this way is to allow a simple lambda function to be passed to `then_execute`:

```
e.then_execute([](T value){return value;}, input_future);
```

The downsides of this design are twofold:

- The description of the continuation is based on ability to invoke it. There is then potential for errors that would easily slip through code review, and silently cause unexpected runtime behaviour.
- The mechanism of describing the continuation with two parallel end-to-end data paths removes the ability to catch and pass an exception from the value operator, or to log and passthrough an exception from the exception operator without rethrowing the exception.

On the first point, consider the following struct that an author might write in an attempt to handle both values and exceptions at some stage in the pipeline:

```
struct callable {  
    int operator()(int value) {  
        return value + 1;  
    }  
    int operator()(std::exception_ptr e) {  
        return 0;  
    }  
};
```

This is a trivial example of ignoring the precise exception and attempting to recover. Note that the reality here, based on the [p0443](#) definition is that the exception function is not callable as the `EXCEPTIONAL` case. It will therefore not be called and an exception will bypass. In effect, this struct is semantically equivalent to:

```

struct callable {
    int operator()(int value) {
        return value + 1;
    }
    int operator()(exception_arg, std::exception_ptr e) {
        std::rethrow_exception(e);
    }
};

```

where we have silently lost our recovery path and passed the error through with potentially negative consequences. There is no compilation or runtime error here, and this kind of problem could be hard to catch in code review.

On the second point, consider an exception handler that only exists to log that an exception reached a point in the stream:

```

struct callable {
    int operator()(int value) {
        return value + 1;
    }
    int operator()(exception_arg, std::exception_ptr e) {
        std::cerr << "Have exception\n";
        std::rethrow_exception(e);
    }
};

```

This is an expensive means of doing nothing to the exception. With potential extensions to `std::exception_ptr` that would allow peeking at the exception without rethrow, for example [p1066](#), there is potentially a wide range of optimisations that we lose the ability to perform.

What we might prefer, would be to implement this as:

```

struct callable {
    int operator()(int value) {
        return value + 1;
    }
    std::exception_ptr operator()(exception_arg, std::exception_ptr e) {
        std::cerr << "Have exception\n";
        return e;
    }
};

```

but then we lose the ability to recreate the value.

We consider these two flaws, one of safety and the other of flexibility, as unfortunate limitations of a low-level API like executors.

Expected use of `FutureContinuation` as discussed in [p1054](#) is through the use of helper functions such as `on_value` and `on_error` that take a constrained callable and return a `FutureContinuation`. With these helper functions, and the clean readable code they lead to, there is no need to simplify the `FutureContinuation` to be a trivial callable, and we gain a lot of flexibility by consciously deciding to not simplify it in that way.

The goal of this paper is to convince the reader that we should solidify the `FutureContinuation` with use cases and examples. We then build on this to demonstrate how bulk execution should be simplified based on similar properties, without committing to a design. Finally we propose wording changes to executors to make the changes concrete.

The same modifications we propose in this paper apply to both [p0443](#) and [p1054](#). Uses of the types in [p1054](#) are unaffected but the description of the calling mechanism, return values of the construction functions (`on_value`, `on_error`) and precise semantics would require updates similar to those we propose for [p0443](#).

§ 2. Requirements

If we look at some example continuation constructions based on those in [p1054](#) we can see what kind of functionality we might want here.

§ 2.1. `then_value`

This is the simple passthrough of the exception, while applying some operation to the value.

The callback we expect to create looks like:

```
then_value([](T value){operation(value);});
```

As a flow diagram, something like:

```
set_value ----- Perform Operation ----- return_value
set_exception ----- return_exception
```

§ 2.2. then_error

The equivalent where we apply some operation to the exception, but not the value. A good example of this might be error recovery. Note that in this case we are breaking the exception chain. The callback we expect to create looks like:

```
then_error([](std::exception_ptr e){
    try {
        std::rethrow_exception(e);
    } catch(recoverable_exception) {
        return 0;
    } catch(...) {
        std::rethrow_exception(std::current_exception());
    }
});
```

Or:

```
set_value -----/----- ret
set_exception ----- Perform Operation ----- | ----- Recover -----/
\----- Do not recover ----- ret
```

Note that in this case we rethrow twice. Logically the first is just to check the exception type. The second is just returning the exception and relying on external logic to catch as we do not have two outputs in the syntax. Improvements to [exception_ptr](#) (along the lines of folly's [exception_wrapper](#) or those proposed in [p1066](#)) could mitigate the first. Ability to return either an [exception_ptr](#) or a [T](#) from the error case could remove the second throw.

§ 2.3. then_variant

Here our operation might take a variant of a value and an exception so that we can write a single function that decides what to do: The callback we expect to create looks like:

```
then_variant([](std::variant<T, std::exception_ptr> v){operation(v);});
```

Diagrammatically:

```
set_value  -----\                               /----- return_value
                |----- Perform Operation -----|
set_exception -/                               \----- return_exception
```

This is a very common pattern in Facebook's code where `folly::Try<T>`, which carries a value and exception, is the preferred means of parameterising future continuations.

§ 2.4. then_value_with_exception_log

Here we merely log the existence of an error, and pass it through. We might write this as:

```
then_value_with_exception_log(
    [](T value){operation(v);},
    [](std::exception_ptr e){std::cerr << "Have an exception\n"; return e;});
```

Here we have a very simple pair of parallel operations:

```
set_value  ----- Perform Operation ----- return_value
set_exception ----- Log ----- return_exception
```

Note though that it relies on allowing return of an `exception_ptr` from the exception path to do this without a throw.

§ 3. Concept

As an alternative way of thinking about this problem we should step back and think about what we want from the solution. Fundamentally, a continuation is a function from a value input or an

exceptional input, to a value output or an exceptional output.

```
set_value  -----\                               /----- return_value
                  |----- Perform Operation -----|
set_exception -/                               \----- return_exception
```

This basic structure covers all of the above uses. The question becomes how we can build this in an efficient manner?

One option is to do what we do in a lot of Facebook's code, and implement the operation in terms of `folly::Try<T>` putting all functionality in the continuation itself. Unfortunately, it is clumsy to write efficient code that wants to ignore one or other path entirely using this structure. We are forced into the combined structure in the code.

Abstractly, though, if we assume that these operations are inputs and outputs from some class, we see that the input is a `Promise` type:

```
                                     /----- return_value
Promise ----- Perform Operation -----|
                                     \----- return_exception
```

Where a promise is a class concept consisting of two `void`-returning functions: `set_value` and `set_exception`.

Taking a further look at this we realise that actually the output path is merely the input to another operation - one owned by the executor itself. So we see another `Promise` lying in wait for us:

```
Promise ----- Perform Operation ----- Promise
```

Fundamentally, then, each of the continuation constructors should produce something that has a promise as input, and a promise as output, and where the value and error operations can be mixed based on the implementation. This is fully general. Moreover, by requiring that both of these functions be provided and thus called by the implementation, it is also safe because the compiler will fail if a function fails to compile. The `set_value` input can map to either the `return_value` or `return_exception` output, and similarly for the `set_exception` input.

So what does this look like? Because the Promises are both concepts, not types, we need to be able to generate this code. The input promise is a feature of the task we construct. This much is simple. In addition, we need a way to take the output promise as an input. That is, we need to construct a usable task from some partial task, plus a promise.

In summary:

The continuation is an object that, when passed a Promise as a parameter constructs a new object that is itself a Promise.

§ 4. Examples

Let's take a few examples of what this looks like to implement.

Given a continuation provided by some continuation construction function (some examples of which we see below) and passed to our processing function, we can use the continuation by: 1) constructing internal `Promise` that is tied to our output `Future` 2) pass the output promise to the continuation as a means of constructing a viable promise object. 3) call the appropriate operation on the input with data from our input `Future`.

In code that general principle looks like:

```
OutputFuture process_continuation(FutureContinuation&& continuation) {
    // Construct output promise/future contract
    [outputPromise, outputFuture] make_promise_contract<T>();

    // Construct the input promise by parameterising the continuation with the
    // output promise.
    auto inputPromise = std::move(continuation)(outputPromise);

    // Call the appropriate input data path on the input promise
    if(have_value()) {
        std::move(inputPromise).set_value(value());
    } else {
        std::move(inputPromise).set_exception(exception());
    }

    // Return the outputFuture that will include the result of the computation
    return outputFuture;
}
```

§ 4.1. then_value

`then_value` takes a function from a value to a value and, as discussed above, returns a function that can be passed a Promise, and which constructs a Promise:

```
// F = function(int(int))
template <typename F>
auto then_value(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
        auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type;

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise))

            void set_value(int value) {
                try {
                    auto resultOfOperation = f_(value);
                    outputPromise_.set_value(resultOfOperation);
                } catch (...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }

            void set_exception(std::exception_ptr e) {
                outputPromise_.set_exception(std::move(e));
            }

        private:
            F f_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(continuationFunction),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}
```

and constructs the continuation as:

```
auto continuation = then_value([](int x) {return x*2;});
```

As an example of using these constructs as if as a simple callback, for an executor that only supports simple callback mechanisms, we can see that all of this code optimises to nothing (<https://godbolt.org/g/m3qv0j>).

§ 4.2. then_error

Here we construct a continuation from a function from `exception_ptr` to `exception_ptr` as a means of only processing our error stream.

```

// F = function(exception_ptr(exception_ptr))
template <typename F>
auto then_error(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
        auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type;

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise))

            void set_value(int value) {
                outputPromise_.set_value(value);
            }

            void set_exception(std::exception_ptr e) {
                try {
                    auto resultOfOperation = f_(std::move(e));
                    // Set the exception from the return value
                    outputPromise_.set_exception(resultOfOperation);
                } catch (...) {
                    // Also catch the error for completeness.
                    outputPromise_.set_exception(std::current_exception());
                }
            }

        private:
            F f_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(continuationFunction),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}

```

We can construct a simple exception processing continuation as:

```
auto continuation = then_error([](std::exception_ptr e) {  
    std::cerr << "Log!\n"; return e;});
```

Note that even here, if we do not end up using the exception path, all of this optimises away (<https://godbolt.org/g/xRm2oH>)

§ 4.3. then_variant

We can implement a version that passes variants through as:

```

// F = function(variant<int, exception_ptr>(variant<int, exception_ptr>))
template <typename F>
auto then_variant(F&& continuationFunction) {
    return [continuationFunction = std::forward<F>(continuationFunction)](
        auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type;

        class InputPromise {
        public:
            InputPromise(
                F&& f, OutputPromise&& outputPromise) :
                f_(std::move(f)), outputPromise_(std::move(outputPromise))

            void set_value(int value) {
                apply(value);
            }

            void set_exception(std::exception_ptr e) {
                apply(std::move(e));
            }

        private:
            F f_;
            OutputPromise outputPromise_;

            void apply(std::variant<int, std::exception_ptr> v) {
                struct visitor {
                    void operator()(int result) {
                        outputPromise_.set_value(std::move(result));
                    }
                    void operator()(std::exception_ptr ex) {
                        outputPromise_.set_exception(std::move(ex));
                    }
                };
                OutputPromise& outputPromise_ = outputPromise_;

                try {
                    auto intermediateValue = f_(std::move(v));
                    std::visit(visitor{outputPromise_}, std::move(intermediateValue));
                } catch (...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }
        };
    };
}

```

```

};

return InputPromise(
    std::move(continuationFunction),
    std::forward<OutputPromiseRef>(outputPromise));
};
}

```

Constructing the continuation with:

```

struct visitor {
    std::variant<int, std::exception_ptr>
    operator()(int val) const {
        return val + 1;
    }

    std::variant<int, std::exception_ptr>
    operator()(std::exception_ptr ex) const {
        return ex;
    }
};

auto continuation = then_variant(
    [](std::variant<int, std::exception_ptr> v) -> std::variant<int, std::exception_ptr> {
        return std::visit(visitor{}, std::move(v));
    });

```

Again, with use of variants, if we do not actually use the `exception_ptr` route this optimises away (<https://godbolt.org/g/AZRAeK>).

§ 4.4. then_value_logging_error

Finally, we can build an operation that takes two functions, where the error handler simply passes through the exception with logging:

```

// F = function(int(int))
template <typename F, typename FE>
auto then_value_log_exception(F&& valueContinuation, FE&& errorContinuation) {
    return [valueContinuation = std::forward<F>(valueContinuation),
            errorContinuation = std::forward<FE>(errorContinuation)](
        auto&& outputPromise) mutable {
        using OutputPromiseRef = decltype(outputPromise);
        using OutputPromise = typename std::remove_reference<OutputPromiseRef>::type;

        class InputPromise {
        public:
            InputPromise(
                F&& f, FE&& fe, OutputPromise&& outputPromise) :
                f_(std::move(f)), fe_(std::move(fe)), outputPromise_(std::move(outputPromise)) {}

            void set_value(int value) {
                try {
                    auto resultOfOperation = f_(value);
                    outputPromise_.set_value(resultOfOperation);
                } catch (...) {
                    outputPromise_.set_exception(std::current_exception());
                }
            }

            void set_exception(std::exception_ptr e) {
                outputPromise_.set_exception(fe_(std::move(e)));
            }

        private:
            F f_;
            FE fe_;
            OutputPromise outputPromise_;
        };

        return InputPromise(
            std::move(valueContinuation),
            std::move(errorContinuation),
            std::forward<OutputPromiseRef>(outputPromise));
    };
}

```


and where we might construct this as:

```
auto continuation = then_value_log_exception(
    [](int x) {return x*2;},
    [](std::exception_ptr e){std::cerr << "Have exception\n"; return e;});
```

Note that with improvements to `exception_ptr` this is where we could benefit from snooping on the exception without rethrow, as `folly::exception_wrapper` enables or is proposed in [p1066](#). Full source example: <https://godbolt.org/g/Xbp5xK>.

§ 5. Noexcept

The methods on `FutureContinuation` should be noexcept. Any exception handling should be handled as part of the `FutureContinuation` task and passed to the `set_exception` output.

With this change, the executors do not need exception propagation properties, nor do they need to expose queries that specify what happens when an exception leaks from a continuation because this cannot happen. This is a considerable simplification and reduction in committee work that is still in progress.

§ 6. Rethinking bulk execution

We should encode bulk operations as extended continuations. Bulk execution is a property of a task, not an executor. While we realise that the executor has influence on how the task runs and where, which may include how it is compiled to do bulk dispatch, the actual properties of the task are orthogonal to the API exposed by executors.

Encoding the bulk functionality as part of the continuation, rather than the executor API, would allow us to halve the number of executor entry points. Further, bulk continuations need not be part of the fundamental concepts and instead we can encode the interface simply as an extended set of task construction functions as in [p1054](#).

We are confident that bulk can be cleanly implemented in this model, but would want to see implementation work done. In our view, implementation work is already necessary to give confidence for the bulk implementations in [p0443](#), particularly for `bulk_then_execute`.

The bulk API can be achieved in multiple ways. `FutureContinuation`'s definition should be extended to map a `Promise` to a `BulkPromise`. The interface of `BulkPromise` offers multiple

options. Let's assume that we base them around a task construction function based on the same parameters of `then_execute` in [p0443](#):

```
FutureContinuation bulk_then_value(F, S, RF, SF);
```

One implementation option is that we allow `set_value` and `set_exception` to be called multiple times, encapsulating the shape and a completion signal in the API of the continuation.

For example:

```
void set_value(int idx, int& value);  
void set_exception(int idx, std::exception_ptr& e);  
const int& get_shape() const;  
void done();
```

Building on earlier examples, this might be implemented more fully as: <https://godbolt.org/g/PPTtrW>. We need `get_shape` to know what iteration domain to call `set_value` over, and `done` because in a parallel use case the continuation itself cannot know when it is complete.

There would be some cost here in deciding repeatedly to skip the exception.

Another option is to say that `set_exception` only be callable once:

```
void set_value(int idx, int& value);  
void set_exception(std::exception_ptr e);  
const int& get_shape() const;  
void done();
```

In this design the implementation can pass an exception through more directly as in <https://godbolt.org/g/gLN72t>.

We might also separate iteration from initialization, and allow the promise to consume its inputs immediately, executing trivially under certain circumstances, for example passing the exception through without practical iteration as above:

```
void set_value(int value);  
void set_exception(std::exception_ptr e);  
const int& get_shape() const;  
void execute_at(int idx);  
void done();
```

This design could be utilized as in <https://godbolt.org/g/RmhaFb>.

Finally, in rethinking bulk in this way we should consider what the result and shared factory functions (RF and SF types in the above) mean, in two specific areas:

- Should the factory functions be independent of the data stream, or should they be part of the data processing operation?
- Should the continuation explicitly translate the result variable into an output?

For the first, we might consider changing the factory function API to be parameterized by the value and the shape. This falls cleanly out of the separated `executor_at` design above, such that the factory functions run when `set_value` and `set_exception` are called. For example, if we take a vector as input, it would make a lot of sense to be able to generate an output that matches the input vector size, based on a size that was computed in the processing stream. If we pass a reference to the input value into the result factory, we can return a vector of that size directly as the result to write to.

The shape can also be considered in this way, and be computed from the input. For example, if we have an exception, return a shape of 1 and we immediately tell the executor that we want to handle the exception as a single stream. If we do not do this we will always execute the full iteration domain, even if all but one instance is going to return directly.

On the second point, output processing at the moment is slightly difficult in that while we can manually perform a reduction operation such that the result variable is a reduced value or exception, we have no function that runs at the end of processing that unpacks this again. Clearly with the above task proposal we can do this as part of the `done()` call - so exposing a means to modify that operation is a powerful primitive.

A very limited implementation of parameterising the result factory can be seen in <https://godbolt.org/g/jDVhPC>.

In practice, if we produce a general bulk task, we do not have to constrain the factory functions in any way - they are all properties of the task and called when the executor performs certain operations. We should only specify a few standard ones that implement reasonable basic semantics.

This is a more flexible and powerful model, and we believe one that is equally easy to understand. There may be concerns about implementation cost. For example, running work in `done()` may be a costly additional operation that needs pre-synchronization to work, after the primary bulk operation completes. In practice, though, given that we have to return the object anyway, and that object might have an arbitrary move constructor, we believe any such concerns would need strong justification.

§ 7. Proposed New Wording for P0443

§ 7.1. `Promise` requirements

A type `P` meets the `Promise` requirements for some value type `T` if an instance `p` of `P` satisfies the requirements in the table below.

Expression	Return Type	Operational semantics
<code>p.set_value(T&&)</code>	void	Defined if <code>T</code> is not void. Completes the promise with a value. Should accept by forwarding reference.
<code>p.set_value()</code>	void	Defined if <code>T</code> is void. Completes the promise with no value.
<code>p.set_exception(std::exception_ptr)</code>	void	Completes the promise with an exception wrapped in a <code>std::exception_ptr</code> .

§ 7.2. `OneWayFutureContinuation` requirements

A type `OFC` meets the `OneWayFutureContinuation` requirements if `OFC` satisfies the requirements of `MoveConstructible` and for an instance `ofc` of `OFC`, `INVOKE(std::forward<OFC>(ofc))` is valid.

§ 7.3. `TwoWayFutureContinuation` requirements

A type `TFC` meets the `TwoWayFutureContinuation` requirements if `TFC` satisfies the requirements of `MoveConstructible` and for an instance `tfc` of `TFC` and a value `p` whose type, `P` satisfies the `Promise` requirements and for which `INVOKE(std::forward<P>(p))` is valid.

§ 7.4. ThenFutureContinuation requirements

A type `THFC` meets the `TwoWayFutureContinuation` requirements if `THFC` satisfies the requirements of `MoveConstructible`, and for an instance `thfc` of `TFC` and a value `p` whose type, `P` that satisfies the `Promise` requirements and where `INVOKE(std::forward<P>(p))` is valid and returns a value `pout` of type `POUT` that satisfies the `Promise` requirements for some value type `TIn`, potentially `void`, that is known to the caller.

§ 7.5. Changes to OneWayExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X` and `f` denotes an object of type `F&&` that satisfies the requirements of `OneWayFutureContinuation`.

§ 7.6. Changes to TwoWayExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `f` denotes a an object of type `F&&` that satisfies the requirements of `TwoWayFutureContinuation` and `R` is `void` or denotes the value type of a value `p` of type `P&&` that satisfies the `Promise` requirements for value type `R` and that may be passed to the expression `DECAY_COPY(std::forward<F>(f))(std::move(p))`.

Expression	Return Type	Operational semantics
<code>p.twoway_execute(f)</code>	A type that satisfies the Future requirements for the value type <code>R</code> .	Creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))(p)</code> for some value <code>p</code> of type <code>P</code> that satisfies the requirements of <code>Promise</code> for value type <code>R</code>, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>twoway_execute</code>. May block pending completion of <code>DECAY_COPY(std::forward(f))(p)</code>. The invocation of <code>twoway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code>. Stores the result of a call to <code>p.set_value(r)</code>, <code>p.set_value()</code>, or <code>p.set_exception(e)</code> for <code>r</code> of type <code>R</code> or <code>e</code> of type

`std::exception_ptr` in the associated shared state of the resulting `Future`.

§ 7.7. Changes to ThenExecutor requirements

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `fut` denotes a future object satisfying the `Future` requirements, `f` denotes a function object of type `F&&` that satisfies the requirements of `ThenFutureContinuation` and `R` is `void` or denotes the value type of a value `p` of type `P&&` that satisfies the `Promise` requirements for value type `R` and that may be passed to the expression `DECAY_COPY(std::forward<F>(f))(std::move(p))`.

Expression	Return Type	Operational semantics
<code>p.then_execute(f, fut)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	When <code>fut</code> is ready, creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))(p)</code> for some value <code>p</code> of type <code>P</code> that satisfies the <code>Promise</code> requirements for value type <code>R</code>, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>then_execute</code>. May block pending completion of <code>DECAY_COPY(std::forward<F>(f))(p)</code>. The invocation of <code>then_execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code>. If <code>fut</code> is ready with a value, calls <code>f.set_value(r)</code>, if <code>fut</code> is ready and the value is void calls <code>f.set_value()</code>. If <code>fut</code> is ready with an exception calls <code>f.set_exception(e)</code>. Stores the result of a call to <code>p.set_value(r)</code>, <code>p.set_value()</code>, or <code>p.set_exception(e)</code> for <code>r</code> of type <code>R</code> or <code>e</code> of type <code>std::exception_ptr</code> in the associated shared state of the resulting <code>Future</code>.

§ 7.8. Changes to Bulk requirements in general

Separate all bulk-related operations into a separate document targeted at a TS.